

Third development changes understanding code block by block

For Page 2 of the frontend, I developed the **Manage Readings** screen, which allows users to create, edit and delete real cold-chain readings through the REST API I built during Week 3. I used **TanStack Query mutations** for the POST, PATCH and DELETE operations rather than manually controlling API request states. The Create Reading form validates the user's data before sending it to the backend, while the backend still performs its own validation for security and reliability. When a reading is created, its UUID and status are generated by the backend, and if the temperature is outside the permitted 2°C–8°C range the frontend displays a clear excursion alert. The Edit Reading section uses PATCH so that the user can update only the fields they want to change, while the Delete Reading section includes an additional confirmation step to reduce accidental deletion. After create, edit or delete operations, I use TanStack Query invalidation so previously cached reading data is marked as out of date. I also created a reusable notification component for success, excursion and error messages, including working close buttons. During DELETE testing I found that the reading was successfully being removed from DynamoDB but the frontend displayed a `500` error. I traced this to the Next.js proxy incorrectly handling the backend's successful `204 No Content` response. I fixed the proxy so a 204 response is returned with a genuinely empty body, after which DELETE worked correctly from the frontend. I also tested backend validation directly using Postman and confirmed that invalid requests were still rejected even when the frontend form was bypassed.

Files involved in Page 2

The main files involved were:

```
1 app/  
2   └─ manage/  
3     └─ page.tsx  
4  
5   └─ components/  
6     └─ AlertNotification.tsx  
7  
8   └─ api/  
9     └─ backend/  
10        └─ [...path]/  
11           └─ route.ts  
12  
13  └─ lib/  
14    └─ api.ts
```

They work together like this:

```
1 /manage  
2   ↓  
3 Create / Edit / Delete
```

```
4   ↓
5   TanStack Query mutations
6   ↓
7   lib/api.ts
8   ↓
9   Next.js proxy
10  ↓
11  REST API
12  ↓
13  Lambda
14  ↓
15  DynamoDB
```

1. `app/manage/page.tsx`

This is the main file for Page 2.

It controls all three write operations:

```
1 | Create → POST
2 | Edit   → PATCH
3 | Delete → DELETE
```

Rather than having three separate pages, I created three modes inside one Manage Readings screen.

"use client"

The file begins with:

```
1 | "use client";
```

This tells Next.js that the page is a **Client Component**.

That is required because the page uses browser-side React functionality such as:

```
1 | useState
2 | forms
3 | buttons
4 | user input
5 | TanStack mutations
```

React imports

We import:

```
1 | import {
2 |   FormEvent,
3 |   useState,
4 | } from "react";
```

`useState` allows the page to remember the values entered into all of the forms.

`FormEvent` gives TypeScript the correct type for form submission events.

For example:

```
1 function handleCreateSubmit(  
2   event: FormEvent<HTMLFormElement>  
3 )
```

tells TypeScript that this function receives an HTML form submission event.

TanStack Query imports

We use:

```
1 import {  
2   useMutation,  
3   useQueryClient,  
4 } from "@tanstack/react-query";
```

There is an important difference between what we used on Page 3 and what we use here.

Page 3 mainly uses:

```
1 useQuery
```

because it **reads** data.

Page 2 mainly uses:

```
1 useMutation
```

because it **changes** data.

So:

```
1 useQuery  
2 GET  
3  
4 useMutation  
5 POST  
6 PATCH  
7 DELETE
```

useQueryClient()

We create:

```
1 const queryClient =  
2   useQueryClient();
```

This gives us access to the TanStack Query cache.

We need this because after changing data, the old cached GET results might no longer be correct.

For example:

```
1 Table cache says:
2 10 readings
3
4 POST creates another reading
5
6 Database now says:
7 11 readings
```

Without invalidation, the interface may continue using the old cache.

`queryClient` lets us tell TanStack Query:

The data has changed. The previous cached result should no longer be considered current.

2. Manage mode

We created:

```
1 type ManageMode =
2   | "create"
3   | "edit"
4   | "delete";
```

This is a TypeScript union type.

It means `mode` can only ever contain:

```
1 create
2 edit
3 delete
```

Then:

```
1 const [mode, setMode] =
2   useState<ManageMode>("create");
```

stores which part of the screen is currently open.

The page starts on:

```
1 Create Reading
```

Mode buttons

The three buttons use:

```
1 setMode("create")
2 setMode("edit")
3 setMode("delete")
```

React then conditionally renders the correct form.

Conceptually:

```
1 [ Create Reading ]
2   ↓
3 mode = create
4   ↓
5 show POST form
6
7
8 [ Edit Reading ]
9   ↓
10 mode = edit
11   ↓
12 show PATCH form
13
14
15 [ Delete Reading ]
16   ↓
17 mode = delete
18   ↓
19 show DELETE form
```

The active button is also visually highlighted.

3. Create Reading state

We created separate state variables for:

```
1 branchId
2 recordedAt
3 minTempC
4 maxTempC
5 recordedBy
```

For example:

```
1 const [
2   branchId,
3   setBranchId,
4 ] = useState("BRANCH-001");
```

The branch starts with:

```
1 BRANCH-001
```

while other form fields initially start empty.

Why Reading UUID is not included

The Create form deliberately does **not** ask the user for:

```
1 readingId
2 status
3 alertRaisedAt
```

because these are managed by the backend.

The frontend sends:

```
1 | branchId
2 | recordedAt
3 | minTempC
4 | maxTempC
5 | recordedBy
```

Then the backend:

```
1 | generates UUID
2 |     ↓
3 | checks temperatures
4 |     ↓
5 | creates status
6 |     ↓
7 | stores reading
```

This maintains the design introduced during the Week 3 backend work.

4. Create form validation

We created:

```
1 | function validatecreateForm()
```

This checks the input before allowing POST.

It checks that:

```
1 | Branch ID exists
2 | Recorded At exists
3 | Minimum temperature is a number
4 | Maximum temperature is a number
5 | Maximum is not below minimum
6 | Recorded By exists
7 | Date/time is valid
8 | Date/time is not in the future
```

For example:

```
1 | if (
2 |     Number(maxTempC) <
3 |     Number(minTempC)
4 | )
```

detects an invalid range.

The frontend then shows:

```
1 | Maximum temperature cannot be lower than minimum temperature.
```

Frontend validation vs backend validation

An important part of the design is that frontend validation does **not replace** backend validation.

We have:

```
1 Frontend validation
2   +
3 Backend validation
```

The frontend provides quick feedback to the user.

The backend remains the final authority.

I proved this using Postman by bypassing the form and sending:

```
1 {
2   "minTempC": 8,
3   "maxTempC": 3
4 }
```

directly to the REST API.

The backend still rejected the invalid request.

That proves someone cannot bypass the rules simply by avoiding the frontend.

5. Create mutation — POST

We created:

```
1 const createMutation =
2   useMutation({
```

This controls the POST operation.

The important section is:

```
1 mutationFn: (
2   input: CreateReadingInput
3 ) =>
4   createReading(input),
```

This tells TanStack Query:

When this mutation is triggered, call `createReading()`.

That function exists in:

```
1 lib/api.ts
```

and eventually sends:

```
1 POST
2 /branches/{branchId}/readings
```

Calling the mutation

After successful validation we use:

```
1 | createMutation.mutate({
```

and provide:

```
1 | branchId
2 | recordedAt
3 | minTempC
4 | maxTempC
5 | recordedBy
```

The temperature input values are originally strings because HTML form inputs return text.

We convert them using:

```
1 | Number(minTempC)
2 | Number(maxTempC)
```

before sending them to the backend.

Converting Recorded At

We use:

```
1 | new Date(
2 |   recordedAt
3 | ).toISOString()
```

This converts the user's local date/time input into the ISO timestamp format expected by the backend.

6. Create loading state

TanStack Query gives us:

```
1 | createMutation.isPending
```

While the POST is happening, the button changes from:

```
1 | Save Reading
```

to:

```
1 | Saving Reading...
```

and the button is disabled.

This prevents users from repeatedly clicking Save while the same request is already processing.

7. Create success state

If the new reading is normal:

```
1 | status = ok
```

the user gets:

```
1 | ✓ Reading saved
```

along with the generated UUID.

For example:

```
1 | Reading 550e8400-... was saved successfully.
```

This confirms that the backend successfully created the reading.

8. Excursion notification

If the backend returns:

```
1 | status = excursion
```

we instead display:

```
1 | △ Excursion detected
2 |
3 | The reading was saved successfully
4 | and an excursion alert has been raised.
```

This is important because it gives the user immediate feedback that the reading is not within the safe range.

It also connects the frontend to the excursion behaviour already implemented by the backend.

9. Create query invalidation

After POST succeeds we run:

```
1 | await queryClient.invalidateQueries({
2 |   queryKey: [
3 |     "readings",
4 |     variables.branchId,
5 |   ],
6 | });
```

This marks the branch reading cache as stale.

For example:

```
1 | POST succeeds
2 |   ↓
3 | DynamoDB changes
4 |   ↓
5 | cached branch readings may now be old
```

```
6      ↓
7  invalidate ["readings", branchId]
8      ↓
9  TanStack knows fresh data is required
```

This is one of the main reasons we use TanStack Query rather than manually handling everything with `useEffect`.

10. Edit Reading state

The Edit form has separate state for:

```
1  editBranchId
2  editReadingId
3  editMinTempC
4  editMaxTempC
5  editRecordedBy
```

The user must identify the existing reading using:

```
1  Branch ID
2  Reading UUID
```

Why Recorded At cannot be edited

Our PATCH endpoint only supports changing:

```
1  minTempC
2  maxTempC
3  recordedBy
```

so those are the only editable fields exposed by the frontend.

We do not pretend the frontend can change fields that the real API does not support.

11. PATCH partial updates

PATCH is different from replacing the whole reading.

The user can leave fields blank.

For example:

```
1  Minimum Temperature:
2  blank
3
4  Maximum Temperature:
5  9
6
7  Recorded By:
8  blank
```

produces approximately:

```
1 {  
2   "maxTempC": 9  
3 }
```

rather than sending empty values for everything else.

Building the PATCH object

We begin with:

```
1 const input:  
2   UpdateReadingInput = {};
```

Then only add fields that contain values.

For example:

```
1 if (editMaxTempC) {  
2   input.maxTempC =  
3     Number(editMaxTempC);  
4 }
```

This is what makes the PATCH genuinely partial.

12. Edit validation

We created:

```
1 function validateEditForm()
```

This checks that:

```
1 Branch ID exists  
2 Reading UUID exists  
3 At least one update field exists  
4 Temperatures are numbers  
5 Max is not below min when both are supplied
```

The form also tells the user:

```
1 Leave a field blank if you do not want to change it.
```

13. Update mutation — PATCH

We created:

```
1 const updateMutation =  
2   useMutation({
```

which calls:

```
1 updateReading(  
2   branchId,
```

```
3   readingId,  
4   input  
5 )
```

The final backend request is:

```
1 PATCH  
2 /branches/{branchId}/readings/{readingId}
```

This uses the PATCH endpoint that was added during the backend development.

14. PATCH cache invalidation

After updating a reading we invalidate two different caches.

First:

```
1 [  
2   "readings",  
3   variables.branchId  
4 ]
```

because the reading inside the branch table may have changed.

Second:

```
1 [  
2   "reading",  
3   variables.branchId,  
4   variables.readingId  
5 ]
```

because the cached individual-reading result may also now be out of date.

We do both using:

```
1 Promise.all([  
2   ...  
3 ])
```

So conceptually:

```
1 PATCH  
2   ↓  
3 Reading changes  
4   ↓  
5 Invalidate branch list  
6 AND  
7 Invalidate individual reading
```

15. Edit success and excursion states

After a normal update we show:

```
1 ✓ Reading updated
```

If the resulting reading is an excursion, we instead show:

```
1 | ▲ Excursion detected
```

This means the user receives feedback based on the actual status returned by the backend.

16. Delete Reading state

The Delete mode stores:

```
1 | deleteBranchId
2 | deleteReadingId
3 | deleteValidationError
4 | showDeleteConfirmation
5 | deletedReadingId
```

These control both the DELETE request and the additional confirmation interface.

17. Delete validation

Before anything can happen:

```
1 | validateDeleteForm()
```

checks that both:

```
1 | Branch ID
2 | Reading UUID
```

exist.

Without them, DELETE is not attempted.

18. Delete confirmation

Unlike the Create and Edit forms, clicking:

```
1 | Delete Reading
```

does **not immediately call the API**.

Instead:

```
1 | setShowDeleteConfirmation(
2 |   true
3 | );
```

displays another box asking:

```
1 | Are you sure you want to permanently delete this reading?
```

and shows the UUID.

The user then has:

```
1 | Cancel
2 | Confirm Delete
```

This was deliberately added because DELETE is destructive.

19. DELETE mutation

Only the:

```
1 | Confirm Delete
```

button calls:

```
1 | confirmDelete()
```

which then triggers:

```
1 | deleteMutation.mutate({
```

The API call eventually becomes:

```
1 | DELETE
2 | /branches/{branchId}/readings/{readingId}
```

20. Delete cache handling

After deletion we invalidate:

```
1 | [
2 |   "readings",
3 |   variables.branchId
4 | ]
```

because the deleted reading should no longer appear in the branch list.

We also use:

```
1 | queryClient.removeQueries({
```

for the individual reading cache.

That is slightly different from invalidation.

For the deleted individual reading:

```
1 | The data isn't merely old.
2 | The reading no longer exists.
```

So removing that cached query makes sense.

21. DELETE 500 bug

During testing I encountered an important bug.

The frontend displayed:

```
1 | 500 Internal Server Error
```

when deleting a reading.

Initially it looked like DELETE itself had failed.

However, when I checked DynamoDB I found that:

```
1 | the reading had actually been deleted
```

I then used Postman to investigate further.

This proved that the backend operation was happening successfully.

What caused the problem?

A successful DELETE could return:

```
1 | 204 No Content
```

A 204 response means:

The request succeeded but there is no response body.

Our Next.js proxy was trying to forward the response in the same way as requests that return JSON.

That caused the proxy to fail even though AWS had successfully performed the deletion.

22. Fixing route.ts

We updated:

```
1 | app/api/backend/[...path]/route.ts
```

to detect:

```
1 | if (response.status === 204)
```

and explicitly return:

```
1 | return new Response(  
2 |   null,  
3 |   {  
4 |     status: 204,
```

```
5   }  
6 );
```

The important part is:

```
1 | null
```

because a `204 No Content` response must genuinely contain no body.

After this change:

```
1 DELETE  
2 ↓  
3 DynamoDB removes reading  
4 ↓  
5 204 No Content  
6 ↓  
7 Next.js correctly forwards 204  
8 ↓  
9 Frontend recognises success  
10 ↓  
11 ✓ Reading deleted
```

This solved the misleading `500` error.

23. AlertNotification.tsx

We created a reusable component for displaying notifications.

It supports:

```
1 success  
2 excursion  
3 error
```

through:

```
1 type AlertNotificationProps = {  
2   type:  
3     | "success"  
4     | "excursion"  
5     | "error";
```

Notification content

Each notification receives:

```
1 title: string;  
2 message: string;
```

For example:

```
1 title="Reading saved"
```

and:

```
1 message="..."
```

This means we can use the same component for many situations instead of creating separate alert designs.

Different notification styles

The component selects styling based on the `type`.

For example:

```
1 success
2   → green styling
3   → ✓
4
5 excursion
6   → warning styling
7   → ⚠
8
9 error
10  → red styling
11  → ✖
```

24. Making the X button work

Originally the notifications visually contained a close button, but clicking it did not remove the notification.

We fixed this by adding:

```
1 onClose?: () => void;
```

to the component props.

The `?` means that the close function is optional.

Then the button uses:

```
1 onClick={onClose}
```

So the parent component decides what should happen when the user closes the message.

Closing validation notifications

For example:

```
1 onClose={() =>
2   setCreateValidationError("")
3 }
```

clears the validation error state.

Because the condition:

```
1 | createValidationError &&
```

is no longer true, React removes the notification.

Closing TanStack Query notifications

For mutation messages we can use:

```
1 | onClose={() =>
2 |   createMutation.reset()
3 | }
```

This resets the TanStack mutation state.

So:

```
1 | isSuccess = true
2 |   ↓
3 | notification displayed
4 |   ↓
5 | user presses ×
6 |   ↓
7 | mutation reset
8 |   ↓
9 | isSuccess no longer true
10 |  ↓
11 | notification disappears
```

The same approach is used for update and error notifications.

Closing DELETE success

The DELETE success message depends on:

```
1 | deletedReadingId
```

so we close it using:

```
1 | setDeletedReadingId("")
```

Once the string is empty, React stops displaying the message.

25. lib/api.ts

Page 2 uses three functions we had already created:

```
1 | createReading()
2 | updateReading()
3 | deleteReading()
```

These correspond directly to:

```
1 POST
2 PATCH
3 DELETE
```

The page itself therefore doesn't contain low-level API URLs or the API key.

Instead:

```
1 /manage
2   ↓
3 createReading()
4 updateReading()
5 deleteReading()
6   ↓
7 lib/api.ts
```

This makes the application easier to maintain.

CreateReadingInput

The API layer defines:

```
1 CreateReadingInput
```

which describes what POST is allowed to receive.

It deliberately excludes:

```
1 readingId
2 status
3 alertRaisedAt
```

because those are backend-controlled.

UpdateReadingInput

The update type contains optional properties:

```
1 minTempC?: number;
2 maxTempC?: number;
3 recordedBy?: string;
```

The `?` matches the behaviour of PATCH because not every field has to be supplied.

26. User-friendly errors

If the API responds unsuccessfully, `lib/api.ts` attempts to read:

```
1 message
```

or:

```
1 | error
```

from the response.

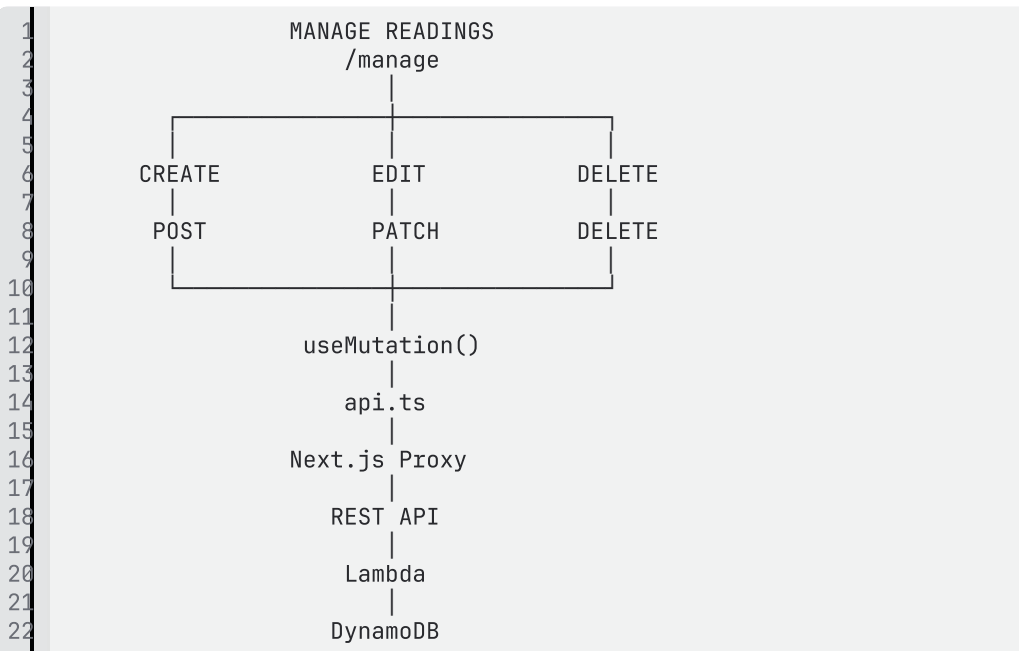
That means Page 2 can show useful validation feedback from the real backend rather than always showing:

```
1 | Something went wrong.
```

This directly supports the ticket requirement to tell the user what actually caused the request to fail.

Final Page 2 architecture

The finished Page 2 now works like:



And after a change:

```
1 | Database changes
2 |   ↓
3 | TanStack Query cache invalidated
4 |   ↓
5 | GET data can be refreshed
```

Page 2 requirements completed

Page 2 now provides:

- ```
1 | ✓ Real backend integration
2 |
3 | ✓ POST – Create Reading
4 | ✓ PATCH – Edit Reading
5 | ✓ DELETE – Delete Reading
6 |
```

```
7 ✓ Server-generated UUID
8 ✓ Backend-generated status
9
10 ✓ Create form validation
11 ✓ Edit validation
12 ✓ Delete validation
13
14 ✓ Frontend validation
15 ✓ Backend validation still enforced
16 ✓ Backend tested directly through Postman
17
18 ✓ TanStack useMutation
19 ✓ Mutation pending/loading states
20
21 ✓ Query invalidation after POST
22 ✓ Query invalidation after PATCH
23 ✓ Query invalidation after DELETE
24
25 ✓ Branch cache invalidation
26 ✓ Individual reading cache handling
27
28 ✓ Excursion notification
29 ✓ Success notifications
30 ✓ API error notifications
31 ✓ Validation notifications
32
33 ✓ Working × close buttons
34
35 ✓ DELETE confirmation screen
36 ✓ Prevents accidental immediate deletion
37
38 ✓ DELETE 204 response bug identified
39 ✓ Frontend 500 error diagnosed
40 ✓ Next.js proxy corrected
41 ✓ Successful DELETE confirmed
```

This means Page 2 is now the **write/manage side of the Cold Chain Operations Portal**, while Page 3 provides the GET/read side.